

Track A — Configurable Options Reference

This document lists **every configurable parameter** available in Track A (Two-Stage STA Baseline) for running different experiments. Track A consists of two stages: **Stage A** (Detector/Candidate Generation) and **Stage B** (Head/Reasoner for semantic prediction).

Table of Contents

1. [YAML Config File \(configs/trackA.yaml \)](#)
 2. [Stage A: Detector Configuration](#)
 3. [Stage B: Head/Reasoner Configuration](#)
 4. [K-Sweep Configuration](#)
 5. [Environment Variable Overrides](#)
 6. [Sweep Scripts Configuration](#)
 7. [Quick Reference Tables](#)
-

1. YAML Config File

File: `local_extraction/configs/trackA.yaml`

The main configuration file for Track A. All settings can be modified here.

Inheritance

```
_base_: "base.yaml" # Inherits shared settings from base.yaml
```

Inherited settings from `base.yaml` :

- `version` : Data version (default: `"v2"`)
 - `paths.*` : Common path definitions
 - `runtime.*` : Runtime settings (progress bars, etc.)
 - `demo.*` : Demo/smoke test settings
-

2. Stage A: Detector Configuration

Stage A generates candidate bounding boxes from frames using either YOLO detection or oracle (ground-truth) mode.

Detection Mode

```
stage_a:
  mode: "yolo" # Options: "yolo", "oracle"
```

Value	Description
"yolo"	Run Ultralytics YOLOv8 inference to generate proposals
"oracle"	Use ground-truth YOLO label files as proposals (for upper-bound testing)

Top-K Candidates

```
stage_a:
  k: 6 # Number of top candidates to keep per image
```

- **Range:** 1-20 (recommended: 4-12)
- **Impact:** Higher K increases recall but lowers precision; more candidates for Stage B

Frame Selection

```
stage_a:
  last_frame_only: true # true: only last frame per clip; false: all frames
  max_images: null      # null = all; integer to cap total images processed
```

Setting	Description
last_frame_only: true	Process only the final frame per UID (faster, typical for STA)
last_frame_only: false	Process all frames (more data, longer runtime)
max_images: null	No limit, process all available images
max_images: 500	Cap at 500 images (for quick testing)

YOLO-Specific Settings

```
stage_a:
  yolo:
    weights:
      "${paths.local_extraction}/toolkit_yolo/runs/sta_yolov8s_singlecls_20251113_00
      2330/weights/best.pt"
    imgsiz: 960 # Input image size for YOLO
```

```
conf_thresh: 0.05 # Confidence threshold (lower = more boxes)
nms_iou: 0.45 # NMS IoU threshold (higher = more overlapping boxes)
```

Parameter	Default	Description
weights	Custom fine-tuned	Path to YOLO weights (<code>.pt</code> file)
imgsz	960	Image size for inference (larger = more accurate but slower)
conf_thresh	0.05	Minimum confidence to keep a detection
nms_iou	0.45	Non-maximum suppression IoU threshold

Typical weight options:

- `yolov8s.pt` — Pre-trained YOLOv8 small (COCO)
- `toolkit_yolo/runs/.../best.pt` — Fine-tuned on Ego4D STA (recommended)

Oracle Mode Settings

```
stage_a:
  oracle:
    label_space: "clips" # Options: "clips", "videos"
```

- `clips` : Labels organized per clip (typical for STA)
- `videos` : Labels organized per video

Output Options

```
stage_a:
  output:
    save_per_image_csv: false # Save individual CSV per image (verbose)
    run_prefix: "trackA_stageA" # Prefix for run folder name
```

3. Stage B: Head/Reasoner Configuration

Stage B processes Stage A candidates: crops ROIs, measures recall, and prepares head manifests for downstream training.

Candidates Source

```
stage_b:
  candidates_source: null # null = auto-detect latest Stage A run
```

- Set to explicit path if needed:
"local_extraction/runs/Track_A/trackA_stageA_20251201_120000/candidates.jsonl"

Cropping Settings

```
stage_b:
  crop_size: [256, 256] # [Width, Height] or null for native size
  keep_top_n: null      # null = keep all candidates; integer to limit
```

Setting	Description
crop_size: [256, 256]	Resize all crops to 256×256 (standardized input)
crop_size: [224, 224]	Standard ImageNet size
crop_size: null	Keep native crop size (variable)
keep_top_n: null	Use all candidates from Stage A
keep_top_n: 5	Keep only top 5 candidates per image

Evaluation Settings

```
stage_b:
  eval_with_labels: true # Compute recall metrics using GT labels
  iou_thresh: 0.5       # IoU threshold for positive match
```

Setting	Description
eval_with_labels: true	Compute recall@K and IoU metrics
iou_thresh: 0.5	Standard IoU threshold (lower = easier matching)
iou_thresh: 0.3	More lenient matching
iou_thresh: 0.75	Stricter matching

Head Manifests

```
stage_b:
  manifests:
    use_clip_manifest: true # true: clip-level; false: video-level
    train: "${paths.manifests_root}/head_train_clip.json"
    val: "${paths.manifests_root}/head_val_clip.json"
```

These manifests contain semantic labels (verb_id, noun_id, ttc) for head training.

Output Options

```
stage_b:
  output:
    write_head_train_val: true # Write merged head_train.jsonl /
head_val.jsonl
    write_semantics_in_manifest: true # Include semantic fields in manifest
rows
    write_back_to_stagea_summary: true # Update Stage A summary with recall
metrics
    run_prefix: "trackA_stageB" # Prefix for run folder name
```

4. K-Sweep Configuration

Automated sweep over multiple K values to find optimal recall-precision tradeoff.

```
k_sweep:
  enabled: false
  k_values: [4, 6, 8, 10, 12, 15]
```

Enable and customize K values for systematic experiments.

5. Environment Variable Overrides

Stage A supports environment variables for quick parameter changes without editing YAML:

Variable	Description	Example
STAGEA_MODE	Override detection mode	oracle , yolo
STAGEA_K	Override top-K value	10
STAGEA_MAX_IMAGES	Cap images processed	500

Variable	Description	Example
STAGEA_DEMO_MODE	Enable demo mode	1 , true
STAGEA_DEMO_N	Demo sample count	50

Usage (PowerShell):

```
$env:STAGEA_MODE = "oracle"
$env:STAGEA_K = "10"
python local_extraction/trackA/trackA_stageA/trackA_stageA.py
```

6. Sweep Scripts Configuration

oracle_k_sweep.py

Runs Stage A (oracle mode) and Stage B for multiple K values automatically.

Variable	Default	Description
ORACLE_SWEEP_KS	"1,2,3,5,8,10"	Comma-separated K values
ORACLE_SWEEP_MAX_IMAGES	500	Images per sweep run

Usage:

```
$env:ORACLE_SWEEP_KS = "4,6,8,10,12"
$env:ORACLE_SWEEP_MAX_IMAGES = "1000"
python local_extraction/trackA/trackA_stageA/oracle_k_sweep.py
```

sweep_k_metrics.py

Advanced K-sweep with detailed metrics, diagnostics, and qualitative analysis.

In-file configuration (edit directly):

```
# ===== CONFIG (edit here) =====
VERSION = "v2"
SPACE = "clips"
K_VALUES = [4, 6, 8, 10, 12, 15]
IOU_THRESHOLD = 0.50

# Matching behavior
CLASS_MATCH_REQUIRED = False # False: IoU-only; True: require class match
```

```
ENABLE_CLASS_MAPPING = False    # Remap predicted classes before matching
CLASS_MAPPING: Dict[int, int] = {} # e.g., {39: 0} COCO→Ego4D

# Frame selection
LAST_FRAME_ONLY = True
MAX_IMAGES = None
DEMO_MODE = False
DEMO_N = 50

# YOLO settings
YOLO_WEIGHTS = "path/to/best.pt"
YOLO_IMGSZ = 960
YOLO_CONF = 0.05
YOLO_IOU = 0.45

# Metrics
F_BETA = 2.0 # F-beta score parameter

# Qualitative analysis
ENABLE_PRED_STATS = True        # Collect prediction statistics
ENABLE_QUALITATIVE = True       # Generate qualitative overlays
QUAL_TOP_K = max(K_VALUES)      # K for visualization
QUAL_SAMPLES = 20               # Frames to sample for overlays
QUAL_RANDOM_SEED = 42           # Reproducibility seed
SMALL_AREA_THRESH = 0.005       # Flag small-object misses
CONF_HIST_BINS = 40             # Confidence histogram bins

# Output
SAVE_PER_IMAGE_SAMPLE = True
SAMPLE_LIMIT = 100
DRY_RUN = False
```

7. Quick Reference Tables

Stage A Parameters

Parameter	Location	Values	Impact
mode	stage_a.mode	"yolo" , "oracle"	Detection method
k	stage_a.k	1-20	Candidates per image
last_frame_only	stage_a.last_frame_only	true/false	Single vs all frames
max_images	stage_a.max_images	null or int	Total image cap
yolo.weights	stage_a.yolo.weights	Path	YOLO model file

Parameter	Location	Values	Impact
yolo.imgsz	stage_a.yolo.imgsz	320-1280	Input resolution
yolo.conf_thresh	stage_a.yolo.conf_thresh	0.01-0.5	Confidence filter
yolo.nms_iou	stage_a.yolo.nms_iou	0.3-0.7	NMS threshold
oracle.label_space	stage_a.oracle.label_space	"clips" , "videos"	GT label organization

Stage B Parameters

Parameter	Location	Values	Impact
candidates_source	stage_b.candidates_source	null or path	Input candidates
crop_size	stage_b.crop_size	[W, H] or null	Crop dimensions
keep_top_n	stage_b.keep_top_n	null or int	Candidates per image
eval_with_labels	stage_b.eval_with_labels	true/false	Compute recall metrics
iou_thresh	stage_b.iou_thresh	0.3-0.75	Positive match threshold
use_clip_manifest	stage_b.manifests.use_clip_manifest	true/false	Manifest granularity

Sweep Parameters

Parameter	Script	Values	Impact
K_VALUES	sweep_k_metrics.py	List of ints	K values to test
IOU_THRESHOLD	sweep_k_metrics.py	0.3-0.75	Matching threshold
CLASS_MATCH_REQUIRED	sweep_k_metrics.py	True/False	Class-aware matching
F_BETA	sweep_k_metrics.py	0.5-2.0	F-score weighting
ENABLE_QUALITATIVE	sweep_k_metrics.py	True/False	Generate visualizations

Example Experiment Configurations

1. Quick Oracle Recall Check


```
# trackA.yaml modifications
stage_a:
  mode: "oracle"
  k: 10
  last_frame_only: true
  max_images: 200
```

2. Full YOLO Inference (Production)

```
stage_a:
  mode: "yolo"
  k: 6
  last_frame_only: true
  max_images: null
  yolo:
    weights: "toolkit_yolo/runs/.../best.pt"
    imgsz: 960
    conf_thresh: 0.05
    nms_iou: 0.45
```

3. High-Recall Configuration

```
stage_a:
  mode: "yolo"
  k: 15
  yolo:
    conf_thresh: 0.01 # Lower threshold = more boxes
    nms_iou: 0.6      # Higher NMS = more overlapping boxes
```

4. K-Sweep Experiment

```
$env:ORACLE_SWEEP_KS = "2,4,6,8,10,12,15"
$env:ORACLE_SWEEP_MAX_IMAGES = "1000"
python local_extraction/trackA/trackA_stageA/oracle_k_sweep.py
```

5. Strict IoU Evaluation

```
stage_b:
  eval_with_labels: true
  iou_thresh: 0.75 # Stricter matching
```

Output Artifacts

Stage A Outputs

- `candidates.jsonl` — One JSON line per image with bounding boxes
- `summary.json` — Run statistics and configuration
- `candidates/` — Per-image CSVs (if `save_per_image_csv: true`)

Stage B Outputs

- `crops/` — Cropped ROI images
- `manifest.csv` / `manifest.jsonl` — Per-crop metadata
- `head_train.jsonl` / `head_val.jsonl` — Merged head manifests
- `summary.json` — Run statistics with recall metrics

Sweep Outputs

- `metrics.csv` — Per-K metrics table
- `summary.json` — Full configuration and results
- `samples.jsonl` — Per-image diagnostic samples
- `qualitative/` — TP/FP/FN overlay visualizations

Files Reference

File	Purpose
<code>configs/trackA.yaml</code>	Main YAML configuration
<code>trackA_stageA/trackA_stageA.py</code>	Stage A detector script
<code>trackA_stageB/trackA_stageB.py</code>	Stage B head prep script
<code>trackA_stageA/oracle_k_sweep.py</code>	Oracle K-sweep runner
<code>trackA_stageA/sweep_k_metrics.py</code>	Advanced K-sweep with diagnostics
<code>trackA_stageB/san_stageB.py</code>	Stage B sanity check report
<code>trackA_plots.py</code>	Plotting utilities
<code>trackA_tests.py</code>	Unit tests